

LLVM

What Is LLVM?

The name "LLVM" originally stood for Low Level Virtual Machine. This is because its intermediate representation (IR) has many properties similar to a virtual machine instruction set.

However, this name is now considered a historical artifact. LLVM is not a virtual machine in the sense of the Java VM or .NET CLR. It's much lower-level and doesn't, for example, have a built-in garbage collector or a specific object model.

So, what is it?

Think of LLVM not as a single tool, but as a collection of modular and reusable compiler and toolchain technologies.

The Core Idea:

At its heart, LLVM is a library. You use these libraries to build compilers, optimizers, just-in-time (JIT) compilers, static analysis tools, debuggers, and more.

The Official Definition:

The LLVM project defines itself as follows:

"The LLVM Project is a collection of modular and reusable compiler and toolchain technologies."

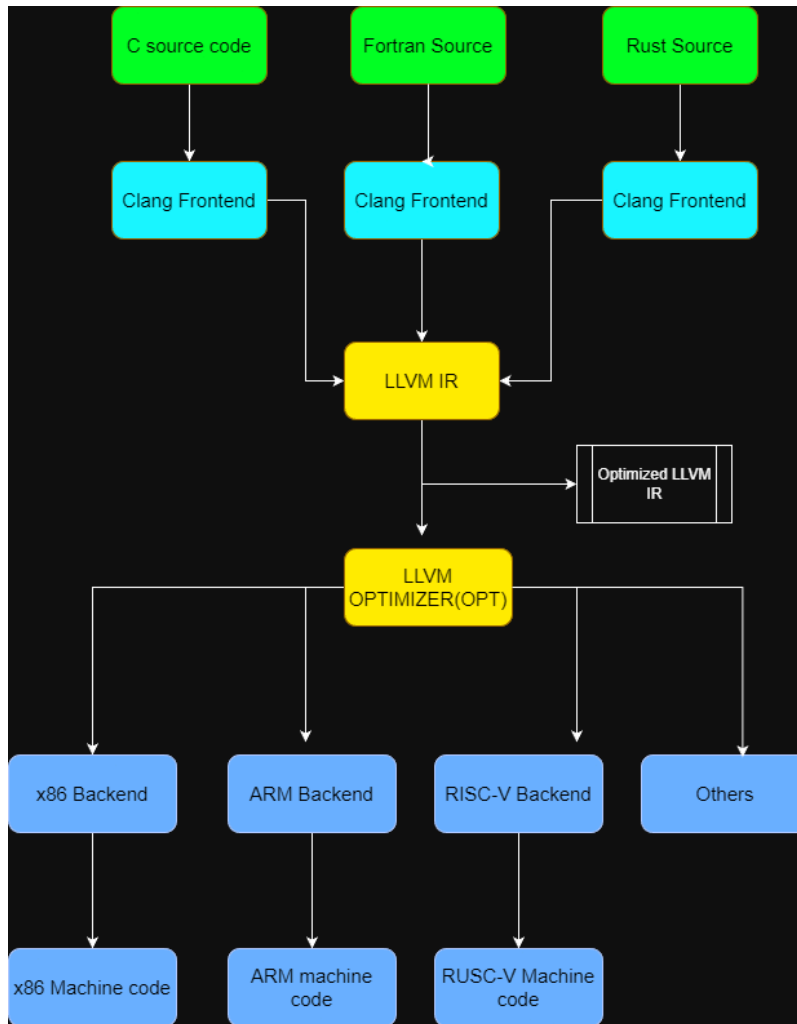
The Three Phase Design

The Traditional Monolithic Compiler Problem

Imagine a world where you have 3 programming languages (C, Fortran, Ada) and 3 different computer architectures (x86, ARM, RISC-V). A traditional compiler is often a single, monolithic program. To support all combinations, you would need to write $3 \times 3 = 9$ separate, complete compilers. This is a huge amount of duplicated effort.

The LLVM Solution: A Pipeline with a Universal Intermediate

LLVM solves this by breaking the compilation process into three distinct stages, with a universal interface between them: the LLVM Intermediate Representation (IR).



Breaking Down the Three Phases:

1. Frontend (The "Language-Specific" Part)

- Job: Translate a specific source language (like C++, Rust, Swift) into the universal LLVM IR.
- What it does: It handles everything specific to the source language: parsing, syntax checking, type checking, and language-specific optimizations.
- Examples: Clang (for C/C++/Objective-C), Flang (for Fortran), the Swift compiler, the Rust compiler (rustc uses LLVM for its backend).

2. Optimizer (The "Target-Independent" Part)

- Job: Take the LLVM IR from the frontend and perform a series of analyses and transformations to make it better—faster, smaller, more efficient.

- The Key: Because all frontends produce the same LLVM IR, the same set of optimizer passes works for every language! You write an optimization once (e.g., "dead code elimination"), and it benefits C, Rust, Fortran, etc.
- Examples of Optimizations: Inlining functions, removing redundant code, simplifying instructions, loop optimizations.

3. Backend (The "Machine-Specific" Part)

- Job: Take the optimized LLVM IR and translate it into the machine code for a specific CPU architecture (like x86, ARM, RISC-V).
- What it does: It handles everything specific to the target hardware: register allocation, instruction selection, scheduling, and platform-specific optimizations.
- The Key: Because all backends consume the same LLVM IR, you can add a new backend to support a new chip, and suddenly every language that uses LLVM can now compile to that new chip.

BENEFIT OF LLVM:

- The Contract: It's the universal agreement between frontends and backends.
- Target-Independent: It doesn't assume a specific CPU or OS.
- Low-Level: It's close to machine code but doesn't commit to specific hardware details.

LLVM Intermediate Representation (IR)

What is LLVM IR?

LLVM IR is a low-level, assembly-like language that is both human-readable and machine-manipulable. It is designed to be a universal, portable representation of code that sits halfway between your source code and the final machine code.

Key Properties of LLVM IR:

1. Static Single Assignment (SSA) Form: This is a critical concept.
 - Rule: Every variable (called a "register" or "value" in IR) is assigned exactly once.
 - Why? This makes data flow analysis much easier for the optimizer. The optimizer can trivially see where a value is defined and where it's used.
 - Example: Instead of `x = x + 1;`, you would see `%x2 = add i32 %x1, 1`. A new "virtual register" (`%x2`) is created for the new value.
2. Strongly Typed: Unlike many assembly languages, every value in LLVM IR has a very specific type (e.g., `i32` for a 32-bit integer, `double` for a double-precision float, `i8*` for a pointer to a byte). This helps the optimizer and backend catch errors and generate better code.
3. Infinite "Virtual Registers": The IR doesn't have a limited set of registers like a physical CPU. It can use as many `%1`, `%2`, `%3`... registers as it needs. The backend's job is to later map these infinite virtual registers onto the finite number of real registers on the target CPU.

Let's Look at Some LLVM IR

Here is a simple C function and its corresponding LLVM IR. This is a perfect place for a code diagram.

C Source Code:

c

```
int add(int a, int b) {  
    return a + b;  
}
```

LLVM IR Generated (simplified):

llvm

```
; Function `add` is defined, taking two i32 (32-bit integer)
parameters, returning an i32.
define i32 @add(i32 %a, i32 %b) {
; A 'basic block' named 'entry'. Every function is a series of basic
blocks.
entry:
; The `add` instruction. It creates a new value `%result` by adding
`%a` and `%b`.
; Note the SSA form: `%result` is assigned once.
%result = add i32 %a, %b

; The `ret` instruction returns the value `%result` from the
function.
ret i32 %result
}
```

Breaking down the structure:

- Module: The top-level container. It holds functions and global variables.
- Function: Defined with define. It has a name (@add), a return type (i32), and a list of parameters ((i32 %a, i32 %b)).
- Basic Block: A sequence of instructions that execute without any branches. It must end with a terminator instruction (like ret, br (branch), or switch). The first block is often labeled entry.
- Instructions: The individual operations (add, ret, load, store, etc.). They operate on the typed values.

A More Complex Example (with control flow):

C Source:

c

```
int max(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

LLVM IR:

llvm

```
define i32 @max(i32 %a, i32 %b) {
entry:
    ; Compare `%a` and `%b` for 'signed greater than'. Result is a
    1-bit integer (i1).
    %comparison = icmp sgt i32 %a, %b
    ; Branch based on the result of the comparison. If %comparison is
    true, go to block `if.then`, else go to block `if.else`.
    br i1 %comparison, label %if.then, label %if.else

if.then:                                     ; preds = %entry
    ret i32 %a

if.else:                                     ; preds = %entry
    ret i32 %b
}
```

(Note: In reality, an optimizer might simplify this, but this shows the raw control flow well).

Why is LLVM IR So Powerful?

- The Universal Interface: It allows any frontend to talk to any backend.
- Optimization-Friendly: Its SSA form and strong typing make it an ideal format for performing complex analyses and transformations.
- Portable: It abstracts away the details of the underlying machine, making it easy to retarget compilers to new CPUs.

LLVM Project Structure

LLVM is not a monolith. It's a large umbrella project containing many subprojects that work together.

```
The LLVM Umbrella Project
|
├── Core LLVM
|   |
|   ├── The LLVM Core Libraries
|   |   (IR, Optimizer, Backend machinery)
|   |
|   └── LLVM Suite Tools
|       (opt, llc, lli, llvm-dis, etc.)
|
├── Major Subprojects (Frontends)
|   |
|   ├── Clang (C/C++/Obj-C Frontend)
|   ├── Flang (Fortran Frontend)
|   └── MLIR (Multi-Level IR, for complex domain-specific languages)
|
├── Major Subprojects (Runtime & Tools)
|   |
|   ├── libc++ (C++ Standard Library)
|   ├── libc++ ABI (Low-level C++ ABI support)
|   ├── compiler-rt (Low-level runtime libraries, e.g., sanitizers)
|   ├── LLDB (Debugger)
|   └── OpenMP (Implementation of the OpenMP standard)
|
└── Major Subprojects (Utilities)
    |
    ├── LLD (Linker)
    └── Polly (High-level loop optimization)
```

1. Core LLVM: The Engine Room

This is what we have been discussing so far.

- The LLVM Core Libraries: This is the collection of C++ libraries that implement all the core functionality: IR data structures, the optimizer passes, all the target backends (x86, ARM, etc.). When you write a tool using LLVM, these are the libraries you link against.

- LLVM Suite Tools: These are command-line tools built using the core libraries. They are both useful utilities and demonstrations of how to use the libraries.
 - opt: The LLVM optimizer. It takes LLVM IR, runs a specified set of optimization passes on it, and outputs the optimized IR.
 - llc: The LLVM static compiler. It takes LLVM IR and compiles it down to assembly code for a specific target CPU (--march=x86, --march=arm, etc.).
 - lli: The LLVM IR interpreter and just-in-time (JIT) compiler. It can directly execute LLVM IR programs.
 - llvm-dis & llvm-as: The disassembler and assembler for LLVM IR. They convert between the human-readable text format (.ll) and the compact binary bitcode format (.bc).

2. Major Subprojects: The Extended Family

- Clang: This is the flagship C, C++, Objective-C, and Objective-C++ frontend for LLVM. It is a production-quality compiler in its own right, known for its fast compile times, excellent error messages, and clang-tidy/clang-format tools. Clang is the program that translates your C++ code into LLVM IR.
- LLD: The LLVM linker. It is designed to be a drop-in replacement for linkers like GNU ld or gold, but much faster.
- LLDB: The LLVM debugger. It is a next-generation, high-performance debugger built as a set of reusable components which greatly benefit from the Clang's expression parser and the LLVM JIT.
- libc++ and libc++abi: LLVM's own implementations of the C++ standard library and the low-level ABI (Application Binary Interface) that supports it. They are designed for high performance and strict standards compliance.
- compiler-rt: Provides highly tuned implementations of low-level library functions (e.g., memcpy, printf) and, crucially, the runtime libraries for LLVM's sanitizer tools (AddressSanitizer/ASan, UndefinedBehaviorSanitizer/UBSan, etc.).
- MLIR: A newer, more ambitious subproject. While LLVM IR is great for mid-level optimizations, MLIR provides a framework for building multiple levels of IR (Multi-Level IR), which is especially useful for complex domains like machine learning, quantum computing, and FPGA design. Think of it as "LLVM's philosophy applied to creating new IRs."

How They Work Together: A Simple Compilation Flow

When you run `clang hello.c -o hello`, here's what happens under the hood (conceptually):

1. Clang parses `hello.c`, does its checks, and generates LLVM IR (in memory or as a `.bc` file).
2. A series of LLVM Core optimization passes are run on this IR.
3. The LLVM Core backend (e.g., the X86 target) is invoked to turn the optimized IR into x86 assembly (`.s` file).
4. The system assembler (`as`) turns the assembly into an object file (`.o`).
5. The system linker (`ld` or `LLD`) links the object file with libraries (like `libc`) to produce the final executable `hello`.

essential command-line tools

We will use a simple C file as our example. Create a file named `example.c`:

```
int factorial(int n) {  
    if (n <= 1)  
        return 1;  
    else  
        return n * factorial(n - 1);  
}
```

Using clang to generate llvm IR

Command:

```
clang -S -emit-llvm example.c -o example.ll
```

- `-S`: Tells clang to emit assembly (in this case, LLVM IR assembly).
- `-emit-llvm`: Specifically tells clang to generate LLVM IR instead of machine assembly.
- `-o example.ll`: Output the result to a file named `example.ll`.

[example.ll](#)

You will get a human-readable LLVM IR file. It will look complex at first, with metadata and declarations, but the core `@factorial` function will be there. It will contain the `define`, `icmp` (integer compare), `br` (branch), `mul` (multiply), and `call` instructions needed to implement the recursive factorial.

Using `opt` - Modular Optimizer

```
opt -S -Oz example.ll -o example_optimized.ll
```

- `-S`: Read input and output in human-readable text form.
- `-Oz`: This is the flag for aggressive size optimization. You could use `-O2` for speed.
- `-o example_optimized.ll`: Output the optimized IR.

Compare `example.ll` and `example_optimized.ll`. The optimized version will likely be very different! The optimizer might:

- Inline the recursive function calls if possible.
- Perform tail-call optimization on the recursion.
- Simplify control flow.
- The code will be transformed into a more efficient, but less obvious, form.

Using `llc` - Static compiler

```
llc -march=x86-64 example_optimized.ll -o example.s
```

- `-march=x86-64`: Specify the target architecture. You could use `-march=arm` for ARM assembly.
- `-o example.s`: Output the assembly code.

You will now get a file containing actual x86-64 assembly code. It will use real CPU registers (`eax`, `edi`), real instructions (`imul`, `cmp`, `je`), and will have the specific syntax for your platform (e.g., AT&T vs. Intel). The abstraction of LLVM IR is now completely gone.

Using `lli` - IR Interpreter/JIT

```
lli main.ll  
echo $?
```

- `lli` loads the IR from `main.ll`.
- It JIT-compiles the IR in memory to native machine code for your host system.
- It executes the `main` function.
- The `echo $?` command on Linux/Mac prints the exit code of the last program, which would be the return value of `main` (in this case, 120, which is 5!).

```
[ example.c ] (C Source Code)
|
| clang -S -emit-llvm
v
[ example.ll ] (LLVM IR - Unoptimized)
|
| opt -Oz
v
[ example_optimized.ll ] (LLVM IR - Optimized)
|               |
| llc -march=x86-64 | lli
v               v
[ example.s ]      [ Program Output ]
(x86 Assembly)    (Executed Result)
|
| System Assembler & Linker
v
[ executable ]
(Native Binary)
```

WeeWoo Lang

WeeWoo is a toy programming language that demonstrates how to build a complete compiler frontend that targets LLVM IR. It features:

- Custom syntax with fun keywords like wee, woo, weewoo
- Basic types: numbers, strings, booleans
- Variables, functions, control flow
- LLVM JIT compilation for immediate execution

WeeWoo Keyword	Traditional Equivalent	Purpose
wee	var	Variable declaration
woo	print	Output statement
weewoo	if	Conditional statement
woowee	else	Else clause
woowoo	def	Function definition
weewoowee	return	Return from function

Example program:

```
woowoo main() {  
  wee x = 10  
  wee name = "Hello World"  
  
  weewoo (x > 5) {  
    woo name  
  } woowee {  
    woo "x is too small"  
  }  
  
  weewoowee 0  
}
```

Architecture overview

```
[ WeeWoo Source ] → [ Lexer ] → [ Parser ] → [ AST ] → [ CodeGen ]  
→ [ LLVM IR ] → [ JIT Execution ]
```

Component 1: Lexer.h/cpp

Convert raw source code into tokens.

- Negative numbers vs. subtraction **operator**
- String literals with escape sequences
- Custom keywords via `m_KeywordMap`
- Comments starting with `'#'`

Component 2: Parser.h/cpp

Pratt Parsing for handling operator precedence.

```
// This is the core of expression parsing  
std::unique_ptr<ExprAST> parseBinOpRHS(int exprPrec,  
std::unique_ptr<ExprAST> lhs)
```

Component 3: ast.h

Represent the program structure in memory.

Key Design Pattern: Visitor Pattern through `virtual codegen()` method

```
class ExprAST {  
public:  
    virtual ~ExprAST() = default;  
    virtual llvm::Value* codegen() = 0; // Each node knows how to  
    generate itself  
};
```

AST Hierarchy

```
ExprAST
├── NumberExprAST, StringExprAST, BoolExprAST (literals)
├── VariableExprAST, VarDeclAST, AssignmentAST (variables)
├── BinaryExprAST (operations)
├── CallExprAST, PrintAST (statements)
├── IfExprAST, BlockAST (control flow)
└── ReturnExprAST (functions)
```

Component 3: codegen.h/cpp

Convert AST to LLVM IR.

1. LLVM Context & Module:

```
std::unique_ptr<llvm::LLVMContext> TheContext;
std::unique_ptr<llvm::Module> TheModule;
std::unique_ptr<llvm::IRBuilder<>> Builder;
```

2.

3. SSA through Memory: Your variables use alloca/load/store instead of PHI nodes, which is easier for beginners:

```
llvm::AllocaInst* Alloca = CreateEntryBlockAlloca(TheFunction,
m_Name, VarType);
Builder->CreateStore(InitVal, Alloca);
```

4.

5. Control Flow: The IfExprAST::codegen() shows perfect LLVM basic block management:

```
llvm::BasicBlock* ThenBB = llvm::BasicBlock::Create(*TheContext,
"then", TheFunction);
llvm::BasicBlock* ElseBB = llvm::BasicBlock::Create(*TheContext,
"else");
llvm::BasicBlock* MergeBB = llvm::BasicBlock::Create(*TheContext,
"ifcont");
```

6.

The IR Builder is your best friend:


```
// Instead of manually creating instructions:  
return Builder->CreateFAdd(L, R, "addtmp"); // So much easier!
```

JIT compilation Integration : main.cpp

```
llvm::EngineBuilder* engineBuilder = new  
llvm::EngineBuilder(std::move(TheModule));  
engineBuilder->setEngineKind(llvm::EngineKind::JIT);  
llvm::ExecutionEngine* EE = engineBuilder->create();  
EE->runFunction(mainFunc, {});
```